

Bài 19: Lambda Expressions trong Java

Lambda Expression là cú pháp ngắn gọn để biểu diễn một hàm không có tên. Lambda thường được dùng với Functional Interface, Collection và Stream API.

1. Functional Interface

Functional Interface là interface chỉ có đúng một abstract method.

Có thể dùng annotation `@FunctionalInterface` để compiler kiểm tra.

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

Interface này có thể được cài đặt bằng lambda:

```
Calculator add = (a, b) -> a + b;

System.out.println(add.calculate(3, 4)); // 7
```

2. Cú pháp Lambda Expression

Cú pháp tổng quát:

```
(tham số) -> biểu thức hoặc khối lệnh
```

Các dạng thường gặp:

```
// Không có tham số
() -> System.out.println("Hello Java");

// Một tham số
name -> System.out.println(name);

// Nhiều tham số
(a, b) -> a + b;

// Nhiều câu lệnh
(a, b) -> {
    int result = a + b;
```

```
    return result;
};
```

Kiểu dữ liệu của tham số thường được Java tự suy luận:

```
Calculator multiply = (a, b) -> a * b;
```

3. Các Functional Interface có sẵn

Các interface phổ biến nằm trong package `java.util.function`.

Interface	Method chính	Mục đích
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Kiểm tra điều kiện
<code>Consumer<T></code>	<code>void accept(T t)</code>	Nhận dữ liệu, không trả về
<code>Function<T, R></code>	<code>R apply(T t)</code>	Chuyển đổi dữ liệu
<code>Supplier<T></code>	<code>T get()</code>	Cung cấp dữ liệu
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	Nhận và trả về cùng kiểu
<code>BinaryOperator<T></code>	<code>T apply(T a, T b)</code>	Xử lý hai giá trị cùng kiểu

4. Predicate

`Predicate<T>` nhận một giá trị và trả về `true` hoặc `false`.

```
import java.util.function.Predicate;

Predicate<Integer> isEven = number -> number % 2 == 0;

System.out.println(isEven.test(4)); // true
System.out.println(isEven.test(5)); // false
```

5. Consumer

`Consumer<T>` nhận một giá trị nhưng không trả về kết quả.

```
import java.util.function.Consumer;

Consumer<String> printer = text -> System.out.println(text);

printer.accept("Hello Lambda");
```

6. Function

`Function<T, R>` nhận kiểu dữ liệu `T` và trả về kiểu dữ liệu `R`.

```
import java.util.function.Function;

Function<String, Integer> stringLength = text -> text.length();

System.out.println(stringLength.apply("Java")); // 4
```

7. Supplier

`Supplier<T>` không nhận tham số nhưng trả về một giá trị.

```
import java.util.function.Supplier;

Supplier<String> message = () -> "Hello from Supplier";

System.out.println(message.get());
```

8. Lambda với Collection

Lambda thường được dùng với `forEach` để duyệt collection.

```
import java.util.List;

List<String> names = List.of("An", "Binh", "Chi");

names.forEach(name -> System.out.println(name));
```

9. Lambda với Comparator

Sắp xếp danh sách theo độ dài chuỗi:

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

List<String> names = new ArrayList<>(List.of("Java", "SQL", "Lambda"));

names.sort((first, second) -> first.length() - second.length());

System.out.println(names); // [SQL, Java, Lambda]
```

Có thể viết dễ đọc hơn:

```
names.sort(Comparator.comparing(String::length));
```

10. Lambda với Stream API

Lambda là thành phần quan trọng của Stream API.

```
import java.util.List;

List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);

numbers.stream()
    .filter(number -> number % 2 == 0)
    .map(number -> number * 2)
    .forEach(System.out::println);
```

Kết quả:

```
4
8
12
```

Ý nghĩa:

- **filter**: lọc dữ liệu theo điều kiện.
- **map**: biến đổi từng phần tử.
- **forEach**: thực hiện hành động với từng phần tử.

11. Method Reference

Method reference là cú pháp rút gọn của lambda khi lambda chỉ gọi một method có sẵn.

```
// Lambda
names.forEach(name -> System.out.println(name));

// Method reference
names.forEach(System.out::println);
```

Các dạng phổ biến:

```
// Static method
Integer::parseInt
```

```
// Instance method của object cụ thể
System.out::println

// Instance method của một kiểu dữ liệu
String::toUpperCase

// Constructor reference
ArrayList::new
```

Ví dụ:

```
Function<String, String> upperCase = String::toUpperCase;

System.out.println(upperCase.apply("java")); // JAVA
```

12. Lambda truy cập biến bên ngoài

Lambda có thể truy cập local variable nếu biến đó là `final` hoặc effectively final.

```
int multiplier = 2;

Function<Integer, Integer> doubleNumber =
    number -> number * multiplier;

System.out.println(doubleNumber.apply(5)); // 10
```

Không được thay đổi giá trị biến sau khi lambda sử dụng biến đó:

```
int multiplier = 2;
Function<Integer, Integer> result = number -> number * multiplier;

// multiplier = 3; // Lỗi
```

13. Lambda và `this`

Trong lambda, `this` tham chiếu đến object bên ngoài.

```
class Printer {
    private String message = "Hello";

    void print() {
        Runnable task = () -> System.out.println(this.message);
        task.run();
    }
}
```

```
}  
}
```

14. Lambda và Anonymous Class

Anonymous class:

```
Runnable task = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running");  
    }  
};
```

Lambda:

```
Runnable task = () -> System.out.println("Running");
```

Lambda chỉ dùng được với Functional Interface. Anonymous class có thể dùng với interface có nhiều method hoặc class abstract.

15. Điểm chính

- Lambda là hàm không có tên, giúp viết code ngắn gọn hơn.
- Lambda cần một Functional Interface làm kiểu đích.
- Functional Interface chỉ có một abstract method.
- **Predicate** dùng để kiểm tra điều kiện.
- **Consumer** nhận dữ liệu nhưng không trả về.
- **Function** chuyển đổi dữ liệu.
- **Supplier** cung cấp dữ liệu.
- Lambda thường kết hợp với Collection và Stream API.
- Method reference là cách viết rút gọn của một số lambda.

16. Thực hành tốt

- Dùng **@FunctionalInterface** khi tự định nghĩa Functional Interface.
- Đặt tên lambda rõ ràng khi logic phức tạp.
- Không viết lambda quá dài; nên tách thành method riêng.
- Dùng method reference khi nó giúp code dễ đọc hơn.
- Tránh thay đổi state bên ngoài lambda.
- Dùng **Comparator.comparing** thay cho phép trừ khi so sánh số để tránh lỗi tràn số.

17. Các lỗi thường gặp

Lỗi	Nguyên nhân
Target type of a lambda conversion must be an interface	Lambda không được gán cho class thông thường
Multiple non-overriding abstract methods	Interface có nhiều abstract method
Local variable must be final or effectively final	Lambda truy cập biến local đã bị thay đổi
Không suy luận được kiểu tham số	Thiếu kiểu đích hoặc lambda không rõ kiểu
Lambda quá dài và khó đọc	Đưa quá nhiều logic vào một biểu thức

18. Ghi nhớ

Lambda = (tham số) -> biểu thức hoặc khối lệnh

Functional Interface + Lambda
→ triển khai hành vi ngắn gọn

Ví dụ tổng quát:

```
@FunctionalInterface
interface Operation {
    int execute(int a, int b);
}

Operation multiply = (a, b) -> a * b;

System.out.println(multiply.execute(3, 4)); // 12
```